

- Anderson, E., et al. 1999, LAPACK User's Guide, 3rd ed. (Philadelphia: S.I.A.M.). Online with software at 2007+, <http://www.netlib.org/lapack>.
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §3.3, and p. 50.
- Forsythe, G.E., and Moler, C.B. 1967, *Computer Solution of Linear Algebraic Systems* (Englewood Cliffs, NJ: Prentice-Hall), Chapters 9, 16, and 18.
- Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §4.1.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.11.
- Horn, R.A., and Johnson, C.R. 1985, *Matrix Analysis* (Cambridge: Cambridge University Press).

2.4 Tridiagonal and Band-Diagonal Systems of Equations

The special case of a system of linear equations that is *tridiagonal*, that is, has nonzero elements only on the diagonal plus or minus one column, is one that occurs frequently. Also common are systems that are *band-diagonal*, with nonzero elements only along a few diagonal lines adjacent to the main diagonal (above and below).

For tridiagonal sets, the procedures of *LU* decomposition, forward- and back-substitution each take only $O(N)$ operations, and the whole solution can be encoded very concisely. The resulting routine `tridag` is one that we will use in later chapters.

Naturally, one does not reserve storage for the full $N \times N$ matrix, but only for the nonzero components, stored as three vectors. The set of equations to be solved is

$$\begin{bmatrix} b_0 & c_0 & 0 & \cdots \\ a_1 & b_1 & c_1 & \cdots \\ & & \cdots & \\ & & \cdots & a_{N-2} & b_{N-2} & c_{N-2} \\ & & \cdots & 0 & a_{N-1} & b_{N-1} \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_1 \\ \cdots \\ u_{N-2} \\ u_{N-1} \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ \cdots \\ r_{N-2} \\ r_{N-1} \end{bmatrix} \quad (2.4.1)$$

Notice that a_0 and c_{N-1} are undefined and are not referenced by the routine that follows.

```
tridag.h void tridag(VecDoub_I &a, VecDoub_I &b, VecDoub_I &c, VecDoub_I &r, VecDoub_O &u)
Solves for a vector u[0..n-1] the tridiagonal linear set given by equation (2.4.1). a[0..n-1],
b[0..n-1], c[0..n-1], and r[0..n-1] are input vectors and are not modified.
{
    Int j,n=a.size();
    Doub bet;
    VecDoub gam(n);           One vector of workspace, gam, is needed.
    if (b[0] == 0.0) throw("Error 1 in tridag");
    If this happens, then you should rewrite your equations as a set of order N - 1, with u1
    trivially eliminated.
    u[0]=r[0]/(bet=b[0]);
    for (j=1;j<n;j++) {       Decomposition and forward substitution.
        gam[j]=c[j-1]/bet;
        bet=b[j]-a[j]*gam[j];
    }
```

```

    if (bet == 0.0) throw("Error 2 in tridag");      Algorithm fails; see below.
    u[j]=(r[j]-a[j]*u[j-1])/bet;
  }
  for (j=(n-2);j>=0;j--)
    u[j] -= gam[j+1]*u[j+1];      Backsubstitution.
}

```

There is no pivoting in `tridag`. It is for this reason that `tridag` can fail even when the underlying matrix is nonsingular: A zero pivot can be encountered even for a nonsingular matrix. In practice, this is not something to lose sleep about. The kinds of problems that lead to tridiagonal linear sets usually have additional properties which guarantee that the algorithm in `tridag` will succeed. For example, if

$$|b_j| > |a_j| + |c_j| \quad j = 0, \dots, N-1 \quad (2.4.2)$$

(called *diagonal dominance*), then it can be shown that the algorithm cannot encounter a zero pivot.

It is possible to construct special examples in which the lack of pivoting in the algorithm causes numerical instability. In practice, however, such instability is almost never encountered — unlike the general matrix problem where pivoting is essential.

The tridiagonal algorithm is the rare case of an algorithm that, in practice, is more robust than theory says it should be. Of course, should you ever encounter a problem for which `tridag` fails, you can instead use the more general method for band-diagonal systems, described below (the `Bandec` object).

Some other matrix forms consisting of tridiagonal with a small number of additional elements (e.g., upper right and lower left corners) also allow rapid solution; see §2.7.

2.4.1 Parallel Solution of Tridiagonal Systems

It is possible to solve tridiagonal systems doing many of the operations in parallel. We illustrate by the special case with $N = 7$:

$$\begin{bmatrix} b_0 & c_0 & & & & & \\ a_1 & b_1 & c_1 & & & & \\ & a_2 & b_2 & c_2 & & & \\ & & a_3 & b_3 & c_3 & & \\ & & & a_4 & b_4 & c_4 & \\ & & & & a_5 & b_5 & c_5 \\ & & & & & a_6 & b_6 \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 \\ r_6 \end{bmatrix} \quad (2.4.3)$$

The basic idea is to partition the problem into even and odd elements, recurse to solve the latter, and then solve the former in parallel. Specifically, we first rewrite equation (2.4.3) by permuting its rows and columns,

$$\begin{bmatrix} b_0 & & & c_0 & & & \\ & b_2 & & a_2 & c_2 & & \\ & & b_4 & & a_4 & c_4 & \\ & & & b_6 & & a_6 & \\ a_1 & c_1 & & & b_1 & & \\ & a_3 & c_3 & & & b_3 & \\ & & a_5 & c_5 & & & b_5 \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_2 \\ u_4 \\ u_6 \\ u_1 \\ u_3 \\ u_5 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_2 \\ r_4 \\ r_6 \\ r_1 \\ r_3 \\ r_5 \end{bmatrix} \quad (2.4.4)$$

Now observe that, by row operations that subtract multiples of the first four rows from each of the last three rows, we can eliminate all nonzero elements in the lower-left quadrant. The price we pay is bringing some new elements into the lower-right quadrant, whose

nonzero elements we now call x 's, y 's, and z 's. We call the modified right-hand sides q . The transformed problem is now

$$\begin{bmatrix} b_0 & & & c_0 & & \\ & b_2 & & a_2 & c_2 & \\ & & b_4 & & a_4 & c_4 \\ & & & b_6 & & a_6 \\ & & & & y_0 & z_0 \\ & & & & x_1 & y_1 & z_1 \\ & & & & & x_2 & y_2 \end{bmatrix} \cdot \begin{bmatrix} u_0 \\ u_2 \\ u_4 \\ u_6 \\ u_1 \\ u_3 \\ u_5 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_2 \\ r_4 \\ r_6 \\ q_0 \\ q_1 \\ q_2 \end{bmatrix} \quad (2.4.5)$$

Notice that the last three rows form a new, smaller, tridiagonal problem, which we can solve simply by recursing. Once its solution is known, the first four rows can be solved by a simple, parallelizable, substitution. For discussion of this and related methods for parallelizing tridiagonal systems, and references to the literature, see [2].

2.4.2 Band-Diagonal Systems

Where tridiagonal systems have nonzero elements only on the diagonal plus or minus one, band-diagonal systems are slightly more general and have (say) $m_1 \geq 0$ nonzero elements immediately to the left of (below) the diagonal and $m_2 \geq 0$ nonzero elements immediately to its right (above it). Of course, this is only a useful classification if m_1 and m_2 are both $\ll N$. In that case, the solution of the linear system by LU decomposition can be accomplished much faster, and in much less storage, than for the general $N \times N$ case.

The precise definition of a band-diagonal matrix with elements a_{ij} is that

$$a_{ij} = 0 \quad \text{when} \quad j > i + m_2 \quad \text{or} \quad i > j + m_1 \quad (2.4.6)$$

Band-diagonal matrices are stored and manipulated in a so-called compact form, which results if the matrix is tilted 45° clockwise, so that its nonzero elements lie in a long, narrow matrix with $m_1 + 1 + m_2$ columns and N rows. This is best illustrated by an example: The band-diagonal matrix

$$\begin{pmatrix} 3 & 1 & 0 & 0 & 0 & 0 & 0 \\ 4 & 1 & 5 & 0 & 0 & 0 & 0 \\ 9 & 2 & 6 & 5 & 0 & 0 & 0 \\ 0 & 3 & 5 & 8 & 9 & 0 & 0 \\ 0 & 0 & 7 & 9 & 3 & 2 & 0 \\ 0 & 0 & 0 & 3 & 8 & 4 & 6 \\ 0 & 0 & 0 & 0 & 2 & 4 & 4 \end{pmatrix} \quad (2.4.7)$$

which has $N = 7$, $m_1 = 2$, and $m_2 = 1$, is stored compactly as the 7×4 matrix,

$$\begin{pmatrix} x & x & 3 & 1 \\ x & 4 & 1 & 5 \\ 9 & 2 & 6 & 5 \\ 3 & 5 & 8 & 9 \\ 7 & 9 & 3 & 2 \\ 3 & 8 & 4 & 6 \\ 2 & 4 & 4 & x \end{pmatrix} \quad (2.4.8)$$

Here x denotes elements that are wasted space in the compact format; these will not be referenced by any manipulations and can have arbitrary values. Notice that the diagonal of the original matrix appears in column m_1 , with subdiagonal elements to its left and superdiagonal elements to its right.

The simplest manipulation of a band-diagonal matrix, stored compactly, is to multiply it by a vector to its right. Although this is algorithmically trivial, you might want to study the following routine as an example of how to pull nonzero elements a_{ij} out of the compact storage format in an orderly fashion.

```
void banmul(MatDoub_I &a, const Int m1, const Int m2, VecDoub_I &x,
           VecDoub_O &b)
```

banded.h

Matrix multiply $\mathbf{b} = \mathbf{A} \cdot \mathbf{x}$, where $\mathbf{A}$ is band-diagonal with m_1 rows below the diagonal and m_2 rows above. The input vector is $\mathbf{x}[0..n-1]$ and the output vector is $\mathbf{b}[0..n-1]$. The array $\mathbf{a}[0..n-1][0..m_1+m_2]$ stores $\mathbf{A}$ as follows: The diagonal elements are in $\mathbf{a}[0..n-1][m_1]$. Subdiagonal elements are in $\mathbf{a}[j..n-1][0..m_1-1]$ with $j > 0$ appropriate to the number of elements on each subdiagonal. Superdiagonal elements are in $\mathbf{a}[0..j][m_1+1..m_1+m_2]$ with $j < n-1$ appropriate to the number of elements on each superdiagonal.

```
{
    Int i,j,k,tmploop,n=a.nrows();
    for (i=0;i<n;i++) {
        k=i-m1;
        tmploop=MIN(m1+m2+1,Int(n-k));
        b[i]=0.0;
        for (j=MAX(0,-k);j<tmploop;j++) b[i] += a[i][j]*x[j+k];
    }
}
```

It is not possible to store the LU decomposition of a band-diagonal matrix $\mathbf{A}$ quite as compactly as the compact form of $\mathbf{A}$ itself. The decomposition (essentially by Crout's method; see §2.3) produces additional nonzero "fill-ins." One straightforward storage scheme is to store the upper triangular factor (U) in a space with the same shape as $\mathbf{A}$, and to store the lower triangular factor (L) in a separate compact matrix of size $N \times m_1$. The diagonal elements of U (whose product, times $d = \pm 1$, gives the determinant) are in the first column of U .

Here is an object, analogous to `LUdcmp` in §2.3, for solving band-diagonal linear equations:

```
struct Banded {
```

banded.h

Object for solving linear equations $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for a band-diagonal matrix $\mathbf{A}$, using LU decomposition.

```
    Int n,m1,m2;
    MatDoub au,al;           Upper and lower triangular matrices, stored compactly.
    VecInt indx;
    Doub d;
    Banded(MatDoub_I &a, const int mm1, const int mm2); Constructor.
    void solve(VecDoub_I &b, VecDoub_O &x); Solve a right-hand side vector.
    Doub det(); Return determinant of A.
};
```

The constructor takes as arguments the compactly stored matrix $\mathbf{A}$, and the integers m_1 and m_2 . (One could of course define a "band-diagonal matrix object" to encapsulate these quantities, but in this brief treatment we want to keep things simple.)

```
Banded::Banded(MatDoub_I &a, const Int mm1, const Int mm2)
: n(a.nrows()), au(a), m1(mm1), m2(mm2), al(n,m1), indx(n)
```

banded.h

Constructor. Given an $n \times n$ band-diagonal matrix $\mathbf{A}$ with m_1 subdiagonal rows and m_2 superdiagonal rows, compactly stored in the array $\mathbf{a}[0..n-1][0..m_1+m_2]$ as described in the comment for routine `banmul`, an LU decomposition of a rowwise permutation of $\mathbf{A}$ is constructed. The upper and lower triangular matrices are stored in `au` and `al`, respectively. The stored vector `indx[0..n-1]` records the row permutation effected by the partial pivoting; d is ± 1 depending on whether the number of row interchanges was even or odd, respectively.

```
{
    const Doub TINY=1.0e-40;
    Int i,j,k,l,mm;
    Doub dum;
    mm=m1+m2+1;
    l=m1;
    for (i=0;i<m1;i++) { Rearrange the storage a bit.
        for (j=m1-i;j<mm;j++) au[i][j-1]=au[i][j];
        l--;
        for (j=mm-l-1;j<mm;j++) au[i][j]=0.0;
    }
}
```

```

d=1.0;
l=m1;
for (k=0;k<n;k++) {                                For each row...
    dum=au[k][0];
    i=k;
    if (l<n) l++;
    for (j=k+1;j<l;j++) {                            Find the pivot element.
        if (abs(au[j][0]) > abs(dum)) {
            dum=au[j][0];
            i=j;
        }
    }
    indx[k]=i+1;
    if (dum == 0.0) au[k][0]=TINY;
    Matrix is algorithmically singular, but proceed anyway with TINY pivot (desirable in
    some applications).
    if (i != k) {                                    Interchange rows.
        d = -d;
        for (j=0;j<mm;j++) SWAP(au[k][j],au[i][j]);
    }
    for (i=k+1;i<l;i++) {                            Do the elimination.
        dum=au[i][0]/au[k][0];
        al[k][i-k-1]=dum;
        for (j=1;j<mm;j++) au[i][j-1]=au[i][j]-dum*au[k][j];
        au[i][mm-1]=0.0;
    }
}
}

```

Some pivoting is possible within the storage limitations of `bandec`, and the above routine does take advantage of the opportunity. In general, when `TINY` is returned as a diagonal element of U , then the original matrix (perhaps as modified by roundoff error) is in fact singular. In this regard, `bandec` is somewhat more robust than `tridag` above, which can fail algorithmically even for nonsingular matrices; `bandec` is thus also useful (with $m_1 = m_2 = 1$) for some ill-behaved tridiagonal systems.

Once the matrix A has been decomposed, any number of right-hand sides can be solved in turn by repeated calls to the `solve` method, the forward- and backsubstitution routine analogous to its same-named cousin in §2.3.

`banded.h`

```

void Bandec::solve(VecDoub_I &b, VecDoub_O &x)
Given a right-hand side vector b[0..n-1], solves the band-diagonal linear equations  $A \cdot x = b$ .
The solution vector  $x$  is returned as x[0..n-1].
{
    Int i,j,k,l,mm;
    Doub dum;
    mm=m1+m2+1;
    l=m1;
    for (k=0;k<n;k++) x[k] = b[k];
    for (k=0;k<n;k++) {                                Forward substitution, unscrambling the permuted rows
        j=indx[k]-1;                                    as we go.
        if (j!=k) SWAP(x[k],x[j]);
        if (l<n) l++;
        for (j=k+1;j<l;j++) x[j] -= al[k][j-k-1]*x[k];
    }
    l=1;
    for (i=n-1;i>=0;i--) {                            Backsubstitution.
        dum=x[i];
        for (k=1;k<l;k++) dum -= au[i][k]*x[k+i];
        x[i]=dum/au[i][0];
        if (l<mm) l++;
    }
}
}

```